

Data Visualization Web Application

Version 2.0 ? AI-Enhanced Data Visualization Dashboard

Submitted By: S. M. Monowar Kayser

University: Daffodil International University

Submission Date: July 10, 2026

Declaration or Submission Information

This report documents Version 2.0 / Update 2 of the Data Visualization Web Application. The system was upgraded with a secure optional AI assistant, improved documentation, tests, and deployment guidance. No private credential is included in the repository.

Acknowledgement

I express sincere gratitude to the course teacher and Daffodil International University for the opportunity to develop a practical, submission-ready data visualization project using Python, Streamlit, Plotly, and AI-assisted analysis.

Executive Summary

Version 2.0 upgrades the Bangladesh Educational Institutes Explorer with an AI Data Assistant that answers natural language questions using summarized dashboard context. The app still supports filters, charts, district-level aggregation, searchable records, and exports. AI configuration is handled through environment variables so secrets remain outside the codebase.

Abstract

This project presents an interactive dashboard for exploring Bangladesh educational institute data from a multi-sheet Excel workbook. The system cleans and standardizes data, visualizes district and category distributions, provides filtered downloads, and adds an optional API-based AI assistant. The assistant is designed to explain observations without fabricating unsupported values.

Table of Contents

The report includes background, objectives, requirements, dataset preparation, architecture, visualization design, AI assistant architecture, prompt strategy, security, implementation, testing, deployment, limitations, and appendices.

List of Figures

Figure 1: Dashboard home screen. Figure 2: District aggregation visualization. Figure 3: Records table. Figure 4: System workflow. Figure 5: Application architecture. Figure 6: AI interaction flow.

List of Tables

Table 1: Technology stack. Table 2: Dataset summary. Table 3: Test cases and results. Table 4: Environment variables.

Introduction

Educational institute datasets are often stored in spreadsheets that are difficult to analyze during presentations. This project transforms the workbook into an interactive web application with filters, charts, summaries, exports, and an AI assistant for guided interpretation.

Background and Motivation

Students, teachers, and evaluators benefit from dashboards that communicate data clearly. The Version 2.0 update adds conversational support so users can ask questions about selected filters, visual patterns, and limitations in plain language.

Problem Statement

Raw Excel files are not ideal for interactive exploration. Users need a secure web-based dashboard that can explain filtered data and visualizations without requiring programming knowledge.

Project Objectives

The objectives are to load and clean the workbook, provide intuitive filters and charts, preserve exact data access through tables and downloads, add secure AI-assisted explanations, and prepare a public GitHub repository with documentation and deployment instructions.

Scope

The project covers descriptive analytics and contextual AI explanations based on summarized dashboard data. It does not perform causal inference, predictive modeling, or exact geospatial point mapping because latitude and longitude are not provided.

Functional Requirements

The app must load Excel sheets, support combined filters, render Plotly visualizations, display records, export CSV/Excel, maintain chat history, build summarized AI context, and handle missing API keys gracefully.

Non-Functional Requirements

The app should be responsive, readable, secure, maintainable, modular, cache data loading, avoid sending unnecessary data to the AI provider, and continue operating when AI is unavailable.

Dataset Description

The dataset is `data/raw/edu_institutes.xlsx`. It includes sheets such as All, Schools, Colleges, Madrasas, Universities, Technical, Professionals, and Teachers Training. Important fields include district, upazila/thana, institute type, level, EIIN, address,

management, MPO, study type, area, and geography.

Data Preparation and Cleaning

The preprocessing module standardizes column names, renames MOBAILE to MOBILE and GEOGRPY to GEOGRAPHY, removes fully empty rows, treats EIIN and mobile values as text, fills missing display values safely, and creates a search field.

System Architecture

The system separates app.py for UI, data_loader.py for workbook loading, preprocessing.py for cleaning and filters, visualizations.py for charts, map_utils.py for district aggregation, ai_context.py for context preparation, ai_service.py for API calls, prompts.py for prompt rules, and config.py for environment settings.

Technology Stack

Python, Streamlit, Pandas, NumPy, Plotly, OpenPyXL, XlsxWriter, Requests, python-dotenv, pytest, python-docx, and ReportLab are used.

Application Workflow

Users select a sheet, apply filters, inspect KPIs and charts, open records, download data, and optionally ask the AI assistant questions. The assistant receives only summarized context from the current dashboard state.

Visualization Design

The interface uses a dark green command-center theme with lime and blue highlights. Charts are intentionally simple: bar charts, donut charts, district bubble aggregation, and comparison views.

AI Assistant Architecture

The AI assistant consists of a Streamlit chat UI, context builder, prompt template, secure configuration loader, and OpenRouter-compatible API client with timeout and error handling.

AI API Selection and Justification

OpenRouter was selected because it provides an OpenAI-compatible chat completion endpoint and a free-model router option. This keeps the integration flexible while allowing API keys to be stored as deployment secrets.

Prompt Engineering Strategy

The system prompt instructs the model to use only provided context, avoid fabricated values, respect filters, distinguish observations from interpretations, avoid unsupported

causation, and protect secrets.

Context Preparation Process

The app summarizes active filters, record counts, unique counts, top districts, institute type distribution, management distribution, MPO distribution, and district aggregation notes. It does not send the complete workbook unnecessarily.

Security and Privacy Measures

No API key is committed. .env is ignored. .env.example contains placeholders only. Deployment secrets must be configured in hosting settings. The assistant cannot access credentials and is instructed not to expose internal prompts.

User Interface and User Experience Design

Version 2.0 adds a visible AI-enhanced label, a dedicated AI Assistant tab, example questions, chat history, clear chat, spinner states, and warning messages when the API key is unavailable.

Implementation Details

The AI service uses requests.post with Authorization bearer token, timeout handling, retry support, and explicit handling for invalid keys, rate limits, empty responses, and provider failures.

Testing Strategy

Automated tests verify workbook loading, missing-column handling, empty filter results, visualization creation, AI context construction, and missing-key behavior. Manual checks verify app startup and UI rendering.

Test Cases and Results

The pytest suite passed: 6 tests passed. Test coverage includes core dashboard and AI integration failure paths. Full live AI API testing requires a valid OPENROUTER_API_KEY configured securely.

ID	Scenario	Expected	Actual	Status
TC-01	Workbook	Loads	Passed	Pass
TC-02	Missing values	Safe	Passed	Pass
TC-03	Empty filter	Safe	Passed	Pass
TC-04	Chart	Generated	Passed	Pass
TC-05	AI context	Built	Passed	Pass
TC-06	Missing key	Warning	Passed	Pass

Deployment Process

Deployment instructions are provided for Streamlit Community Cloud and Render. The required app file is `app.py`. `OPENROUTER_API_KEY` must be added through deployment secrets to activate the AI assistant.

Performance Considerations

Workbook loading is cached with Streamlit. AI context is summarized to reduce payload size. Charts use aggregated data. The assistant avoids sending full raw records unless future changes explicitly require it.

Limitations

The app cannot provide exact institution locations because coordinates are unavailable. AI responses require a valid API key. Without an API key, the assistant displays setup guidance while all visualization features continue to work.

Future Improvements

Future improvements include official district GeoJSON, expanded tests, multiple AI provider support, chart image export, user-uploaded datasets, and more detailed statistical modules.

Conclusion

Version 2.0 delivers a more interactive, educational, and secure data visualization dashboard. It combines traditional BI-style exploration with context-aware AI assistance while preserving transparency and data limitations.

References

Streamlit documentation, Pandas documentation, Plotly documentation, OpenRouter API documentation, OpenPyXL documentation, Requests documentation, and course materials.

Appendices

Appendices include source code, dataset, screenshots, diagrams, deployment guides, changelog, user guide, and tests.